RAPPORT DE VEILLE • PROJET INDIVIDUEL

Contrôler la qualité des données

Étude comparative de Great Expectations, Soda Core et PyDeequ / Deequ

Revue documentaire, protocole reproductible, résultats expérimentaux et preuve de concept de quarantaine

Abdoulaye Bocoum
Sujet 09
19 octobre 2026

Question étudiée

Comment choisir et évaluer un outil de validation de données lorsque les règles sont communes, mais que les moteurs et les sorties diffèrent ? Ce rapport confronte les documentations officielles à un benchmark local, puis examine un mécanisme de décision et de quarantaine construit pour le projet.

Ce document rend compte d’un travail individuel. Les données utilisées sont synthétiques; les résultats et recommandations sont circonscrits au protocole décrit ci-après.

RÉSUMÉ

Résultats principaux

Les chiffres résument les essais effectivement enregistrés; les interprétations restent attachées à leur contexte.

Périmètre	Résultat observé
Jeu expérimental	50 000 commandes synthétiques; une version propre et une version altérée; graine 20261018.
Contrôles comparés	12 règles réparties entre complétude, unicité, validité et cohérence.
Détection	180 comparaisons agrégées règle–outil–passage concordantes avec les comptes attendus.
Durée totale médiane	Soda Core : 0,459 s; PyDeequ / Deequ : 5,322 s; Great Expectations : 10,198 s.
Preuve de quarantaine	À seuil strict : 2 800 lignes invalides isolées et lot bloqué; zéro ligne publiée.

Dans ce protocole, les trois intégrations retrouvent les volumes attendus pour chacune des douze règles. Soda présente le temps total médian le plus court sur l’environnement mesuré. Les artefacts sauvegardés de Soda sont aussi les plus lisibles selon une grille appliquée par une seule personne. Ces observations ne suffisent pas à désigner un gagnant général : les outils utilisent des moteurs différents et le benchmark compare les scripts complets, y compris le chargement et la préparation.

La démonstration de quarantaine est un validateur autonome du projet. Elle contrôle les lignes, bloque par défaut en présence d’anomalies et conserve les données rejetées avec les règles en échec. Elle n’est pas intégrée aux sorties natives des trois bibliothèques ni à une chaîne de déploiement.

Conclusion de l’étude : les résultats soutiennent un choix conditionnel au moteur et au flux de données. Ils ne démontrent ni une supériorité universelle, ni une aptitude à la production.

Pour lire les résultats correctement

La concordance vérifiée porte sur des nombres agrégés. Comme les adaptateurs du benchmark n’ont pas conservé les identifiants fautifs remontés par chaque outil, aucune précision ni aucun rappel au niveau des lignes n’est calculable pour cette comparaison. Les mesures de durée portent sur cinq passages par outil; la médiane réduit l’effet d’un passage extrême sans décrire à elle seule toute la variabilité.

1 INTRODUCTION

Contexte et problématique

La validation de données relie des règles métier explicites à des décisions d’exploitation.

Une donnée exploitable doit satisfaire des conditions qui dépendent du processus : une commande a un identifiant, un montant cohérent avec ses quantités, une devise admise et un état compatible avec ses événements de paiement ou d’expédition. La vérification ponctuelle dans un tableur aide à repérer un défaut, mais elle ne garantit ni la répétabilité du contrôle ni la traçabilité des décisions lorsque le volume grandit.

Les outils étudiés traduisent des attentes sur les données en contrôles exécutables. Ils s’insèrent cependant dans des environnements différents. Great Expectations propose des attentes et des suites de validation; Soda Core utilise un langage déclaratif de contrats et des contrôles adaptés à la source; Deequ exprime des contraintes et calcule des métriques à l’aide de Spark. L’intérêt de la comparaison est donc pratique : vérifier les mêmes règles sur un jeu commun, puis observer les contraintes et la richesse des résultats que chaque intégration produit.

La question de recherche est formulée ainsi : dans quelle mesure Great Expectations, Soda Core et PyDeequ / Deequ retrouvent-ils les défauts connus d’un jeu synthétique partagé, et quels éléments mesurés aident à orienter un choix selon le moteur et le besoin de diagnostic ? Une seconde question porte sur la gestion du lot : peut-on bloquer la publication et isoler les lignes invalides dans un pipeline démonstratif reproductible ?

Le travail est mené individuellement. Il comprend une veille documentaire sur des sources primaires, un jeu de données généré avec vérité terrain, cinq exécutions de chaque outil, une lecture des artefacts effectivement sauvegardés et un prototype de quarantaine. Il ne s’agit ni d’un audit de données d’entreprise ni d’un essai de performance en production.

Objectifs et périmètre

Objectif	Indicateur retenu	Hors périmètre
Évaluer les comptes	Comptage par règle rapproché de la vérité terrain.	Identité des lignes remontées par GX, Soda et Deequ.
Comparer les durées	Médiane du temps total sur cinq passages.	Classement universel ou extrapolation à un cluster.
Examiner les sorties	Grille descriptive appliquée aux fichiers conservés.	Comparaison complète des interfaces et rapports natifs.
Démontrer le routage	Publication, blocage et quarantaine des CSV.	Intégration au moteur métier, CI/CD ou production.

2 MÉTHODE DE VEILLE

Sources et critères de sélection

La veille privilégie les documents primaires permettant de vérifier une fonction ou une contrainte technique.

La collecte documentaire s’est concentrée sur les pages officielles des trois projets, leurs dépôts de code et la publication scientifique fondatrice utilisée pour situer Deequ. Les pages de configuration et de format de résultat éclairent l’écriture des contrôles et l’accès aux diagnostics; les dépôts et métadonnées de paquets servent à vérifier les versions et les éléments de distribution. La veille initiale a été arrêtée au 18 octobre 2026; les références ont été revérifiées le 19 octobre 2026.

Une source a été retenue si elle répondait directement à une question du projet : comment les règles sont-elles exprimées ? Quels moteurs ou connecteurs sont documentés ? Quelles formes de résultat sont disponibles ? Quels éléments sont associés au paquet réellement épinglé ? Les affirmations de capacité dans le rapport sont rattachées à ces sources. Les pages commerciales générales, les comparatifs non sourcés et les billets de tiers n’ont pas été utilisés pour établir les faits techniques.

Source primaire	Usage dans l’analyse	Précaution
Documentation Great Expectations [1–5]	Expectations, connexions, contexte, formats de résultats et actions.	La possibilité d’obtenir des lignes inattendues ne signifie pas que l’export du benchmark les a conservées.
Documentation Soda [7–9]	Contrats, sources et intégration de tests.	Le benchmark n’évalue que le paquet Soda Core 4.27.0 avec DuckDB.
Dépôt Deequ et article [11–15]	Contraintes Spark, suggestions, DQDL et contexte scientifique.	La fonction de sortie DQDL par ligne n’a pas été testée ici.
PyPI et fichiers de licence [10–12]	Métadonnées du paquet et vérification des licences publiées.	Les métadonnées du paquet Soda et le dépôt courant ne coïncident pas entièrement.

La sélection vise à documenter les caractéristiques nécessaires à l’expérience, pas à produire une revue systématique exhaustive. Les pages produit évoluent; leur description est donc datée. Les résultats mesurés du rapport proviennent des fichiers du dépôt, non des documentations des éditeurs.

Précaution sur les licences

Les métadonnées PyPI de soda-core 4.27.0 déclarent « Proprietary », tandis que le dépôt Soda Core courant publie un fichier de licence différent. Ce constat documentaire ne tranche pas les conditions applicables à un paquet particulier. Toute intégration réelle doit vérifier le fichier exact distribué et les conditions associées avec les responsables compétents.

3 OUTILS ÉTUDIÉS

Modèles fonctionnels et contextes

Les trois solutions offrent des mécanismes de validation; elles n'imposent ni la même syntaxe ni le même moteur.

Dimension	Great Expectations	Soda Core	PyDeequ / Deequ
Expression	Expectations regroupées en suites et évaluations.	Contrat déclaratif et checks, notamment sous forme YAML.	Contraintes et métriques appliquées à des DataFrames Spark.
Moteur du test	Pandas pour le benchmark; d'autres sources sont documentées.	DuckDB pour l'essai; connecteurs liés à la source choisie.	Spark local[1] dans le benchmark.
Résultats	Formats configurables; détail des valeurs inattendues possible selon le format.	Résultats de checks et métriques structurées.	Résultat de contrainte et métriques; fonctions dépendent de la version.
Usage qui ressort ici	Organisation des attentes en Python et des validations.	Contrôles déclaratifs sur le moteur SQL testé.	Contrôle de grands DataFrames dans un environnement Spark existant.
Réserve observée	L'export expérimental ne garde que statut et nombre; préparation mesurée notable.	Les temps dépendent du CSV chargé et de DuckDB en mémoire.	Le démarrage Spark pèse dans le temps total local.

Great Expectations

La documentation décrit une attente comme une assertion vérifiable et une suite comme un ensemble cohérent d'attentes [1]. Les formats de résultat sont configurables, du statut booléen au niveau complet; certains réglages permettent de retourner des lignes ou indices inattendus [4]. Le benchmark utilise néanmoins un export réduit. La note de lisibilité obtenue plus loin reflète cet export et non l'ensemble des fonctions natives.

Soda Core

Soda formalise les contrôles dans un contrat et les exécute contre une source déclarée [7–9]. L'essai compare Soda Core 4.27.0 avec DuckDB en mémoire. Les fichiers sauvegardés sont faciles à lire dans ce scénario, mais ils ne constituent pas une évaluation de toutes les interfaces ou fonctions de suivi.

PyDeequ / Deequ

Deequ exprime des contraintes et calcule des métriques au-dessus de Spark [11,14]. PyDeequ fournit l'accès Python utilisé dans l'expérience. Les suggestions de contraintes documentées sont des propositions à examiner; elles ne remplacent pas une définition métier [13]. Une fonction de diagnostic par ligne décrite dans la documentation courante n'a pas été intégrée ni vérifiée dans les versions testées [15].

4 PROTOCOLE EXPÉRIMENTAL

Environnement et exécution

Le scénario commun sert à mesurer une intégration complète dans chaque outil, avec ses propres étapes de préparation.

Une seule famille de données — des commandes de boutique en ligne — est utilisée afin de garder la vérité terrain maîtrisable. Chaque outil reçoit le même CSV altéré et les mêmes intentions de contrôle. Les scripts traduisent toutefois ces intentions dans des moteurs distincts : Pandas, DuckDB et Spark. L'unité de comparaison est donc l'exécution du script du projet, pas un plan d'exécution strictement identique entre moteurs.

Composant	Configuration retenue
Système	macOS 15.8.1, x86_64; 12 processeurs logiques; 32 Gio de mémoire; Java 21.
Langage / outils	Python 3.12.14; Great Expectations 1.24.0; Soda Core 4.27.0; PyDeequ 1.7.0; Spark 4.1.1.
Entrée	CSV de 50 000 lignes; copie propre et copie contenant les anomalies injectées.
Répétitions	Cinq par outil; médiane des durées utilisée comme résumé principal.
Détection	Nombre de lignes rapporté par contrôle comparé au nombre attendu dans la vérité terrain.
Chronométrage	Chargement, préparation lorsqu'elle est isolée, validation et total.

La durée totale comprend ce que le script doit réellement faire dans cette configuration. Pour GX, une phase de préparation est mesurée séparément; pour Deequ, la session Spark locale est arrêtée entre les passages tandis que la passerelle Java reste active. La séparation des phases n'est donc pas parfaitement symétrique. Cette décision donne un temps opérationnel du scénario mais n'isole pas le coût pur de chaque moteur.

La génération vérifie que le jeu propre ne contient aucun défaut détecté et que les injections du jeu altéré correspondent à une vérité terrain conservée. Les anomalies ne se chevauchent pas entre règles, ce qui facilite l'attribution des échecs et simplifie le jeu par rapport à des données de production.

> **La mesure répond à la question « combien de temps prend cette implémentation complète sur ce poste ? ». Elle ne répond pas à « quel moteur est le plus rapide à architecture et plan identiques ? ».**

5 DONNÉES ET VALIDATION

Un jeu documenté et douze règles

La graine fixe et le fichier de vérité terrain permettent de reproduire les injections et leurs décomptes.

Le jeu comprend onze colonnes : dix champs fonctionnels de commande et un identifiant technique stable. Il est généré à partir d'une base valide de 50 000 lignes, puis altéré avec des défauts contrôlés. Deux cents lignes sont associées à chacune des douze règles, sauf les contrôles U1 et U2 d'unicité, qui signalent les deux lignes de chaque doublon et touchent ainsi 400 lignes par règle. Les fichiers expected_violations.csv et le manifeste décrivent l'association entre l'identifiant et la règle.

ID	Dimension	Règle appliquée	Lignes attendues
C1	Complétude	customer_id renseigné	200
C2	Complétude	order_date renseignée	200
C3	Complétude	total_amount renseigné	200
U1	Unicité	order_id renseigné et unique	400
U2	Unicité	payment_reference non vide et unique	400
V1	Validité	quantity entier strictement positif	200
V2	Validité	unit_price positif ou nul, au plus deux décimales	200
V3	Validité	currency dans EUR, USD ou GBP	200
V4	Validité	status dans le domaine prévu	200
K1	Cohérence	total_amount égal à quantity × unit_price, à 0,01 près	200
K2	Cohérence	référence de paiement compatible avec status	200
K3	Cohérence	date d'expédition compatible avec status	200

Les valeurs ont été fabriquées pour tester des règles ciblées; elles ne représentent pas une clientèle réelle. Le jeu ne modélise ni la fréquence naturelle des défauts, ni les corrélations complexes, ni les changements de schéma. Il documente cependant les variables de génération, les règles appliquées et les anomalies attendues, ce qui rend le scénario vérifiable.

Le jeu et ses règles sont documentés dans benchmark/README.md; le protocole et le générateur sont fournis avec le code source.

6 RÉSULTATS

Concordance des détections

Les comptes sauvegardés correspondent aux volumes attendus dans les douze contrôles et les cinq passages.

Règle	Attendu	GX	Soda	Deequ
C1	200	200	200	200
C2	200	200	200	200
C3	200	200	200	200
U1	400	400	400	400
U2	400	400	400	400
V1	200	200	200	200
V2	200	200	200	200
V3	200	200	200	200
V4	200	200	200	200
K1	200	200	200	200
K2	200	200	200	200
K3	200	200	200	200

La comparaison comporte 12 règles × 5 passages × 3 outils, soit 180 comparaisons de comptes règle-exécution concordantes. Les deux règles d’unicité représentent 800 des 2 800 couples ligne-règle attendus; les dix autres représentent 2 000. Le comptage adopte le nombre de lignes concernées pour les doublons, et non le seul excédent au-delà du premier enregistrement.

Interprétation

Le résultat établit que les agrégats des scripts correspondent aux décomptes connus sur ce jeu. Il ne prouve pas que GX, Soda et Deequ ont identifié exactement les mêmes identifiants. Deux ensembles différents peuvent avoir la même cardinalité. Les métriques de précision et de rappel exigeraient les sorties fautives au niveau des lignes, qui n’ont pas été conservées par les adaptateurs du benchmark.

Le résultat doit aussi être rapproché de la structure volontairement simple du jeu : défauts attribués sans chevauchement et règles explicites. Un jeu réel peut contenir plusieurs défauts par ligne, des valeurs ambiguës ou des problèmes de définition métier. La concordance observée est donc un contrôle de fidélité de l’expérience, pas une validation complète des outils pour un domaine métier.

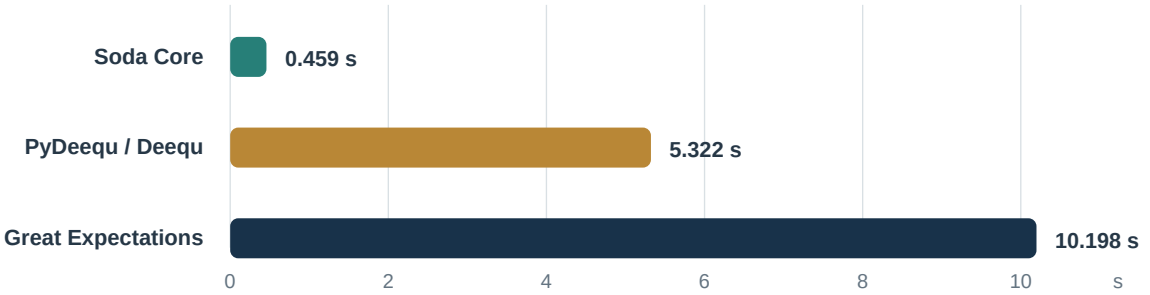
Résultat établi : 180/180 comptes agrégés concordants. Résultat non établi : précision et rappel par enregistrement.

7 PERFORMANCE

Durées observées et phases mesurées

Médianes sur cinq passages; les résultats incluent des étapes d’initialisation propres aux scripts.

Temps total médian pour traiter 50 000 lignes (secondes)



Outil	Chargement	Préparation	Contrôles	Total
Great Expectations 1.24.0	0,194	7,524	2,426	10,198
Soda Core 4.27.0	0,350	n.d.	0,103	0,459
PyDeequ 1.7.0 / Spark 4.1.1	2,387	n.d.	2,421	5,322

n.d. : étape non isolée séparément par le script.

Sur cette configuration locale, le temps total médian de Soda est environ 11,6 fois plus court que celui de Deequ et 22,2 fois plus court que celui de GX. Le contraste chez GX est associé à une préparation mesurée de 7,524 secondes. Pour Deequ, le chargement comprend un coût Spark visible. Le résultat de Soda est quant à lui dominé par le chargement du CSV plutôt que par l’exécution des règles.

Ces valeurs ne représentent pas un benchmark isolant le moteur. Les différences d’architecture, le coût de préparation, la session Spark, les bibliothèques et la manière d’exprimer les contrôles participent toutes à la durée. Un autre volume, une source préchargée ou un environnement distribué pourrait modifier les proportions et le classement.

À utiliser comme ordre de grandeur pour reproduire le scénario local; à ne pas présenter comme une comparaison générale de performance des produits.

8 LECTURE DES SORTIES

Ce que les artefacts conservés montrent

La note descriptive compare les fichiers issus du benchmark; elle ne mesure pas les rapports natifs complets.

Outil	Clarté	Détail	Historique	Export	Moyenne
Great Expectations	3	2	3	4	3,00 / 5
Soda Core	4	4	3	4	3,75 / 5
PyDeequ / Deequ	3	3	3	4	3,25 / 5

Les fichiers Soda montrent un nom de contrôle, un résultat et des métriques comme le nombre et le taux d’erreurs. Deequ conserve une contrainte et une fraction conforme; l’information reste plus technique. L’export GX du benchmark retient un statut et un nombre d’erreurs. Or la documentation GX expose plusieurs formats de résultat, dont des formats pouvant restituer davantage d’éléments inattendus [4].

Cette nuance modifie la portée de l’évaluation : un export plus succinct peut recevoir une note plus faible sans que le produit soit intrinsèquement moins capable de détailler une validation. La grille a été appliquée par une seule personne aux artefacts JSON et Markdown enregistrés. Aucun rapport HTML natif, aucune capture d’interface et aucune comparaison entre évaluateurs n’ont été produits.

La moyenne de Soda (3,75/5) caractérise donc la lisibilité des sorties sélectionnées dans ce projet. Elle est utile pour choisir ce que le pipeline devra conserver, mais elle ne permet pas de départager les interfaces, l’historique, les notifications ou les fonctions d’exploration complètes.

Piste d’amélioration de la mesure

Pour une comparaison plus équitable, il faudrait sauvegarder le rapport natif de chaque outil avec un niveau de détail commun, vérifier la présence d’identifiants stables, puis faire noter les mêmes scénarios par plusieurs personnes au moyen d’une grille définie avant l’inspection. Les critères et extraits utilisés devraient être publiés avec les notes.

9 PIPELINE DE CONTRÔLE

Blocage, publication et quarantaine

Le prototype rend explicite la décision prise après validation et préserve les enregistrements rejetés.

Scénario	Décision	Publiées	Éléments conservés
CSV propre; seuil 0	PASS	50 000	Aucune anomalie; résumé d’exécution.
CSV altéré; seuil 0	BLOCKED	0	2 800 lignes invalides et 2 800 couples ligne-règle; concordance avec la vérité terrain.
CSV altéré; seuil 2 800	PASS	47 200	Les lignes invalides restent en quarantaine; seuil illustratif égal au compte détecté.

Le script applique les douze règles sur chaque ligne et compare le total des enregistrements invalides au seuil configuré. Le comportement par défaut est strict : toute ligne invalide empêche la publication. run_summary.json conserve la décision, les seuils et les comptes. quarantine.csv garde les données écartées et les règles associées; violations.csv liste les couples identifiant-règle; valid_candidate.csv conserve les enregistrements conformes proposés. published.csv n’est produit que lorsque la décision l’autorise.

Le seuil à 2 800 sert uniquement à montrer comment une politique modifie la décision. Il n’est pas recommandé comme seuil métier. Une tolérance doit tenir compte de la criticité des champs, des volumes, des conséquences d’une publication partielle et des responsabilités de correction. Dans un processus réel, un passage avec erreurs devrait rester traçable et pouvoir être réconcilié après correction.

La comparaison du scénario altéré avec la vérité terrain donne 2 800 associations attendues retrouvées, sans faux positif ni faux négatif selon l’implémentation de référence sur ce même jeu. Cette preuve est utile pour valider le routage; comme le détecteur est codé à partir des mêmes règles projet, elle ne constitue pas une validation indépendante du métier.

La preuve porte sur un validateur autonome et des CSV synthétiques. Elle n’est pas raccordée aux sorties par ligne de GX, Soda ou Deequ.

10 ANALYSE CRITIQUE

Limites et validité des conclusions

Les limites concernent autant le jeu et les mesures que la manière de conserver et noter les sorties.

Validité du jeu

Un seul domaine synthétique, un seul format CSV et un volume de 50 000 lignes sont évalués. Le scénario n'inclut ni données sensibles, ni dérive temporelle, ni valeurs ambiguës, ni mélange de plusieurs défauts sur un même enregistrement. Il donne une vérité terrain exacte, mais son caractère construit peut favoriser des règles simples.

Validité de la comparaison des moteurs

Pandas, DuckDB en mémoire et Spark local répondent à des modèles d'exécution différents. Le total inclut le chargement et les coûts d'initialisation propres à chaque script. Cinq passages permettent de calculer une médiane, mais ne suffisent pas à caractériser les distributions; pour Deequ, l'étendue mesurée est importante. Les résultats doivent être repris sur l'architecture visée.

Validité des mesures de détection et de lisibilité

Les agrégats concordants ne révèlent pas les erreurs de substitution à compte égal. La grille qualitative repose sur une seule évaluation de fichiers exportés par le projet; elle ne porte pas sur les fonctions natives complètes. Dans les deux cas, les données conservées limitent ce qu'il est possible de conclure.

Validité opérationnelle

La quarantaine n'a été testée ni sur une base métier ni dans un flux automatisé. Le projet ne couvre pas les alertes, les reprises, les responsabilités de résolution, le suivi dans le temps, les contrôles d'accès, les risques de fuite d'information dans les rejets ni le coût de conservation. Une décision de blocage peut protéger l'aval tout en interrompant un service; cette conséquence doit être évaluée.

> **Aucun résultat de ce rapport n'autorise à affirmer qu'un outil est plus exact, plus sûr ou plus adapté à la production sans essai supplémentaire.**

11 RECOMMANDATIONS

Choisir selon le contexte d’intégration

Les recommandations sont conditionnelles aux configurations documentées et mesurées dans cette étude.

Contexte envisagé	Piste à étudier	Vérification préalable
Données déjà dans un moteur SQL proche de DuckDB; priorité à des checks déclaratifs.	Soda Core constitue le premier candidat à éprouver.	Confirmer connecteur, licence du paquet exact, visibilité des résultats et comportement en cas d’échec.
Pipeline déjà bâti sur Spark; données traitées comme DataFrames distribués.	PyDeequ / Deequ évite d’introduire un moteur distinct pour ce contrôle.	Mesurer l’initialisation et le volume sur le cluster réel; vérifier la sortie d’identifiants ligne par ligne.
Équipe Python qui souhaite organiser des attentes et validations.	Great Expectations mérite un essai avec un résultat suffisamment détaillé.	Configurer et conserver les résultats inattendus, puis mesurer séparément préparation et validation.

Pour la prochaine itération

Le premier travail à mener est d’exécuter les mêmes règles sur un échantillon représentatif de la source réellement visée, en précisant les identifiants métier et la politique attendue face aux erreurs. Il faut ensuite sauvegarder les lignes rejetées par les trois moteurs et mesurer les vrais positifs et faux négatifs, puis refaire les mesures de durée après préchargement lorsque cela correspond au déploiement envisagé.

Une seconde étape consiste à conserver les rapports natifs de chaque produit et à les faire examiner avec une grille commune. Enfin, le prototype de quarantaine pourra être relié à la source de données et au mécanisme de publication : blocage strict par défaut, seuil explicite et justifié, journalisation, contrôle d’accès et scénario de reprise après correction.

Conclusion

Les essais montrent que les trois outils peuvent retrouver les comptes des règles sur ce jeu synthétique. Soda présente le temps médian le plus faible et les artefacts projet les mieux notés, tandis que Deequ et GX correspondent à des écosystèmes et des modes d’organisation distincts. Ces observations donnent des candidats à tester, pas un classement absolu. Le choix final devra s’appuyer sur des identifiants fautifs, des rapports natifs, des durées mesurées dans l’architecture cible et une politique opérationnelle de traitement des rejets.

Recommandation générale : faire dépendre le choix du moteur déjà en place, puis le confirmer par un essai sur des données et des flux représentatifs.

ANNEXE TECHNIQUE

Reproduire et vérifier les livrables

Le dépôt conserve les données, les scripts et les sorties qui étayent les résultats décrits dans le rapport.

Éléments du dépôt

Chemin	Rôle
Protocole_experimentation_qualite_donnees.md	Décrit l'expérience, les règles et les mesures.
benchmark/generate_dataset.py	Génère les CSV, les anomalies et la vérité terrain.
benchmark/data/	Jeu propre, jeu altéré, expected_violations.csv et manifeste.
benchmark/implementations/run_benchmark.py	Exécute les contrôles comparatifs des trois outils.
benchmark/results/full_benchmark.json	Conserve les mesures brutes des cinq passages.
benchmark/implementations/run_quality_gate.py	Validateur autonome, décision, publication conditionnelle et quarantaine.
benchmark/results/quarantine_demo/	Sorties de la démonstration du pipeline.

Commandes principales

Depuis la racine du dépôt :

```
python3 benchmark/generate_dataset.py
python3 benchmark/implementations/run_benchmark.py
python3 benchmark/implementations/run_quality_gate.py --input benchmark/data/orders_50000_clean.csv
--output-dir benchmark/results/quarantine_demo/clean
python3 benchmark/implementations/run_quality_gate.py --input benchmark/data/orders_50000.csv --truth
benchmark/data/expected_violations.csv --output-dir benchmark/results/quarantine_demo/corrupt
```

La dernière commande renvoie le code 2 quand le seuil strict bloque le lot, conformément au comportement attendu. La variante avec seuil 2 800 est illustrée dans la vidéo; le résultat et les fichiers conservés sont dans benchmark/results/quarantine_demo/complete_demo/threshold_2800/.

Code source et livrables

Le code source est disponible dans le dépôt GitHub DataCheckup. Le point d'entrée documentaire est README.md; la description des données et des règles figure dans benchmark/README.md. Le diaporama, la vidéo de démonstration et la transcription sont dans presentation/.

Les versions et l'environnement reproduisent l'essai local décrit. Des dépendances plus récentes peuvent modifier les résultats ou les interfaces.

BIBLIOGRAPHIE

Sources documentaires

Sources primaires consultées le 18 octobre et revérifiées le 19 octobre 2026. Les numéros entre crochets renvoient aux appels dans le texte.

Sources du projet

Les résultats empiriques proviennent des artefacts livrés avec le code : protocole, générateur, vérité terrain, résultats JSON, analyse qualitative, recommandations et exécutions du validateur. Ces fichiers permettent de vérifier les nombres rapportés sans attribuer aux éditeurs les résultats du benchmark.

Références

[1] Great Expectations. “Define Expectations”. Documentation GX Core.

[2] Great Expectations. “Connect to Data”. Documentation officielle.

[3] Great Expectations. “Create a Data Context”. Documentation officielle.

[4] Great Expectations. “Choose a Result Format”. Documentation officielle.

[5] Great Expectations. “Create a Checkpoint with Actions”. Documentation officielle.

[6] Great Expectations. Site officiel et informations générales sur le projet. greatexpectations.io.

[7] Soda. “Contract Language Reference”. Documentation Soda.

[8] Soda. “Data Source Reference for Soda Core”. Documentation Soda.

[9] Soda. “Data Testing”. Documentation Soda.

[10] Python Package Index. soda-core 4.27.0, métadonnées du paquet. PyPI.

[11] Soda Core. Dépôt officiel, incluant le fichier LICENSE du dépôt courant. GitHub. Ne tranche pas les termes applicables au paquet épinglé.

[12] AWS Labs. Deequ, dépôt officiel et informations de licence. GitHub.

[13] AWS Labs. “Constraint Suggestion Example”. Documentation du dépôt Deequ. GitHub.

[14] Schelter, S. et al. (2018). “Automating Large-Scale Data Quality Verification”. *Proceedings of the VLDB Endowment*, 11(12), 1781–1794. DOI: 10.14778/3229863.3229867. Article.

[15] AWS Labs. “DQDL and processRows”. Documentation du dépôt Deequ. Dépôt officiel. La fonction de diagnostic par ligne n’a pas été testée dans les versions du benchmark.

Fin du rapport